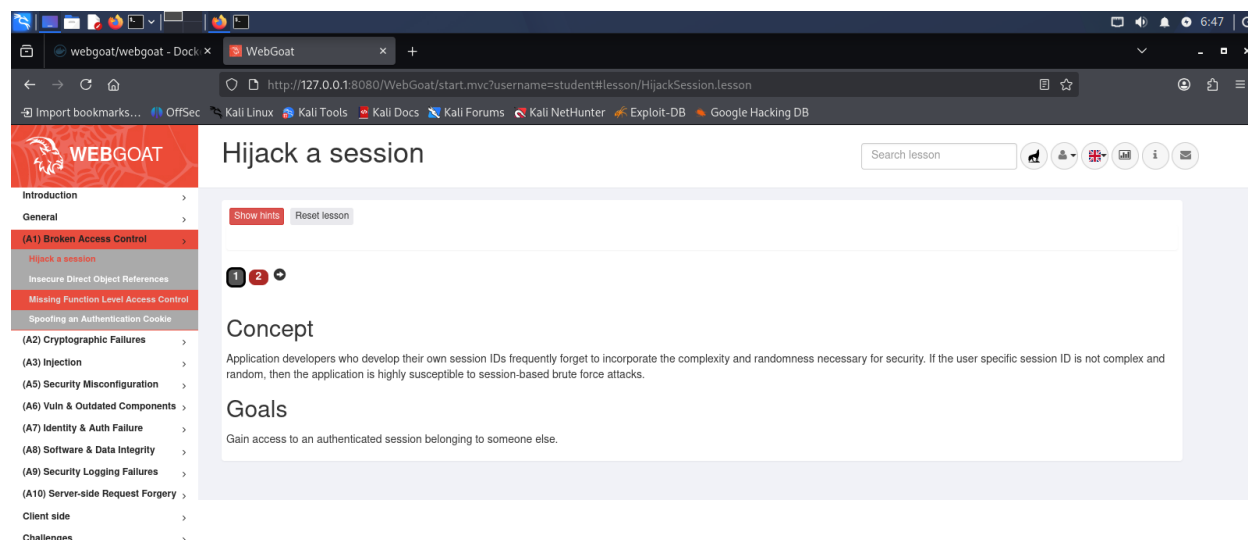


Мета: Знайомство з A01:2025 Broken Access Control

Середовище: Kali Linux, Docker engine, OWASP WebGoat container.

Для кращого розуміння потрібно пройти попередні кроки з мануала WebGoat

В меню обираємо:



<http://127.0.0.1:8080/WebGoat/start.mvc?username=student#lesson/MissingFunctionAC.lesson>

Предивляємось постановку завдання:

Концепція

Розробники прикладного програмного забезпечення, які створюють власні ідентифікатори сесій (session IDs), часто забувають забезпечити рівень складності та рандомізації, необхідний для безпеки. Якщо специфічний ідентифікатор сесії користувача не є складним і випадковим, додаток стає надзвичайно вразливим до атак типу «brute force» (перебір) на сесії.

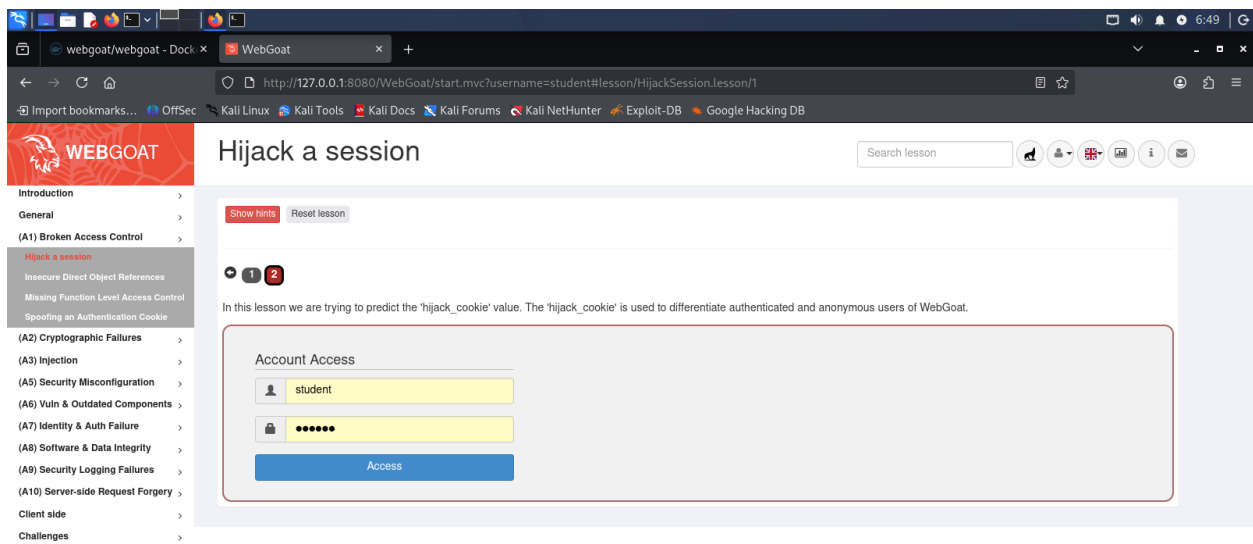
Цілі

Отримати доступ до автентифікованої сесії, що належить іншому користувачу.

Основні терміни:

- **Session ID** — ідентифікатор сесії.
- **Complexity and randomness** — складність та випадковість.
- **Brute force attacks** — атаки методом грубої сили (перебору).
- **Authenticated session** — автентифікована сесія (сеанс).

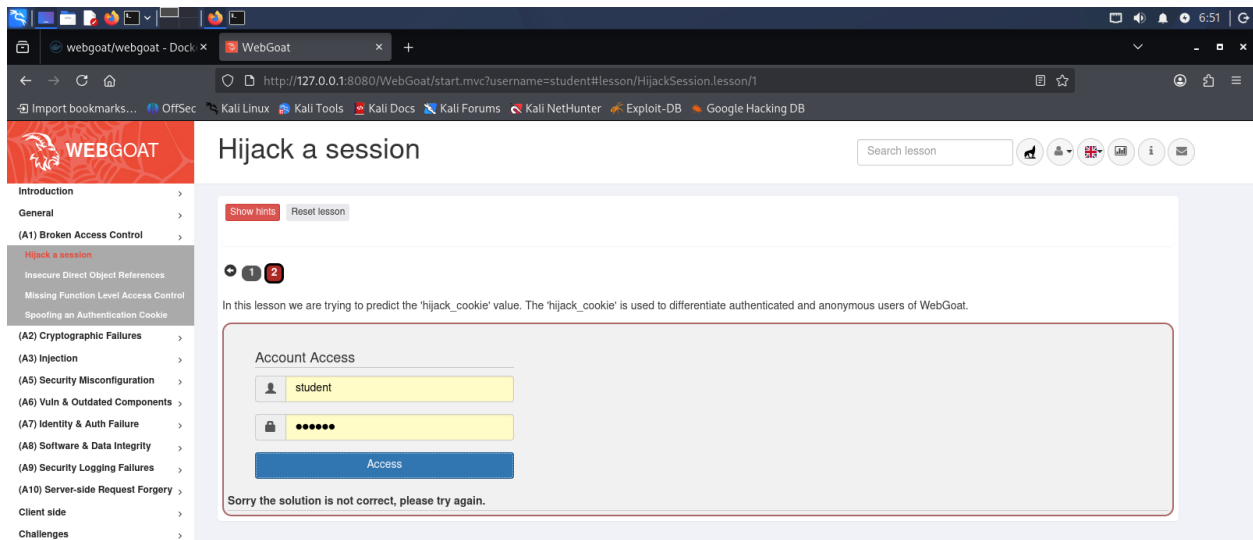
Далі переходимо на другу вкладинку (червоний колір означає, що потрібно буде щось зробити і це буде перевірятись)



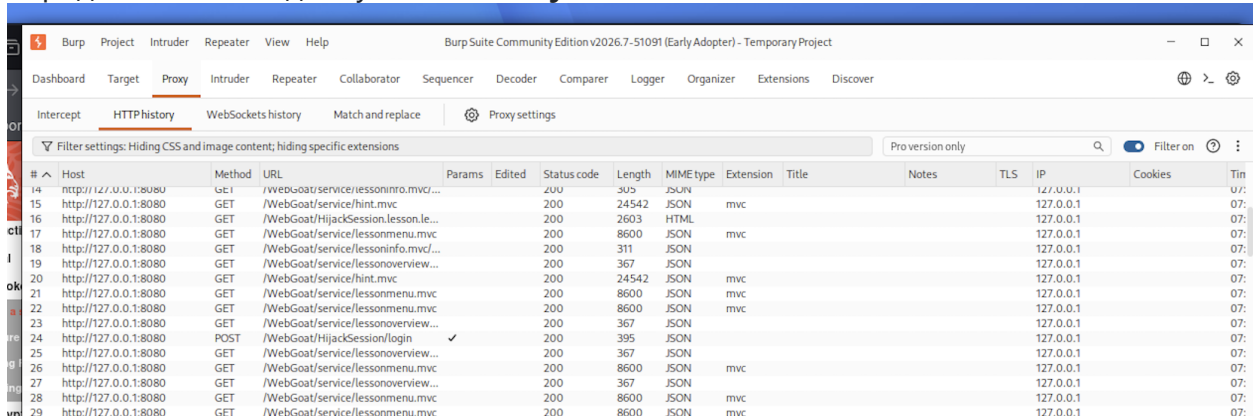
У цьому уроці ми намагаємося передбачити значення «hijack_cookie». Файл «hijack_cookie» використовується для розрізнення автентифікованих та анонімних користувачів WebGoat.

Спробуємо дослідити за допомогою Burpsuit. Запускаємо його, переходимо на вкладинку "Proxy", запускаємо браузер, в якому відкриваємо "WebGoat" та

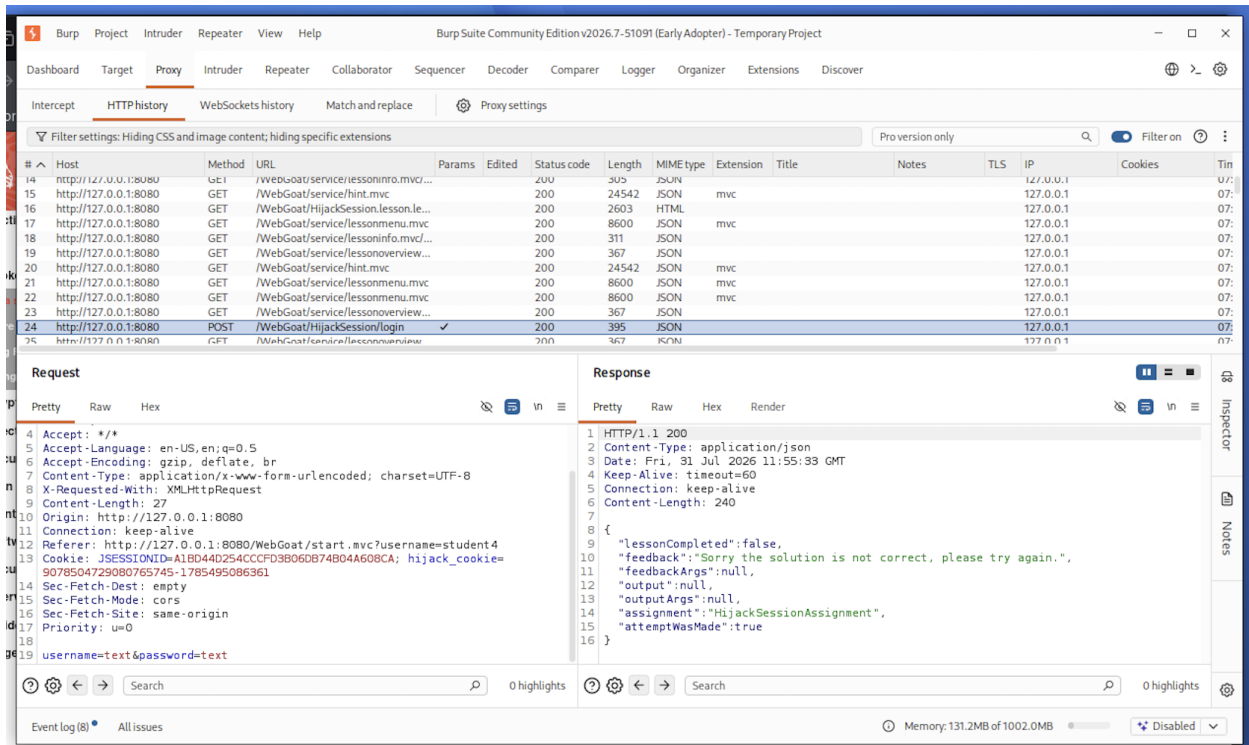
створюємо користавача і логінуємся або одразу логінуємось (якщо не видаляли контейнер) та переходимо на другий крок:



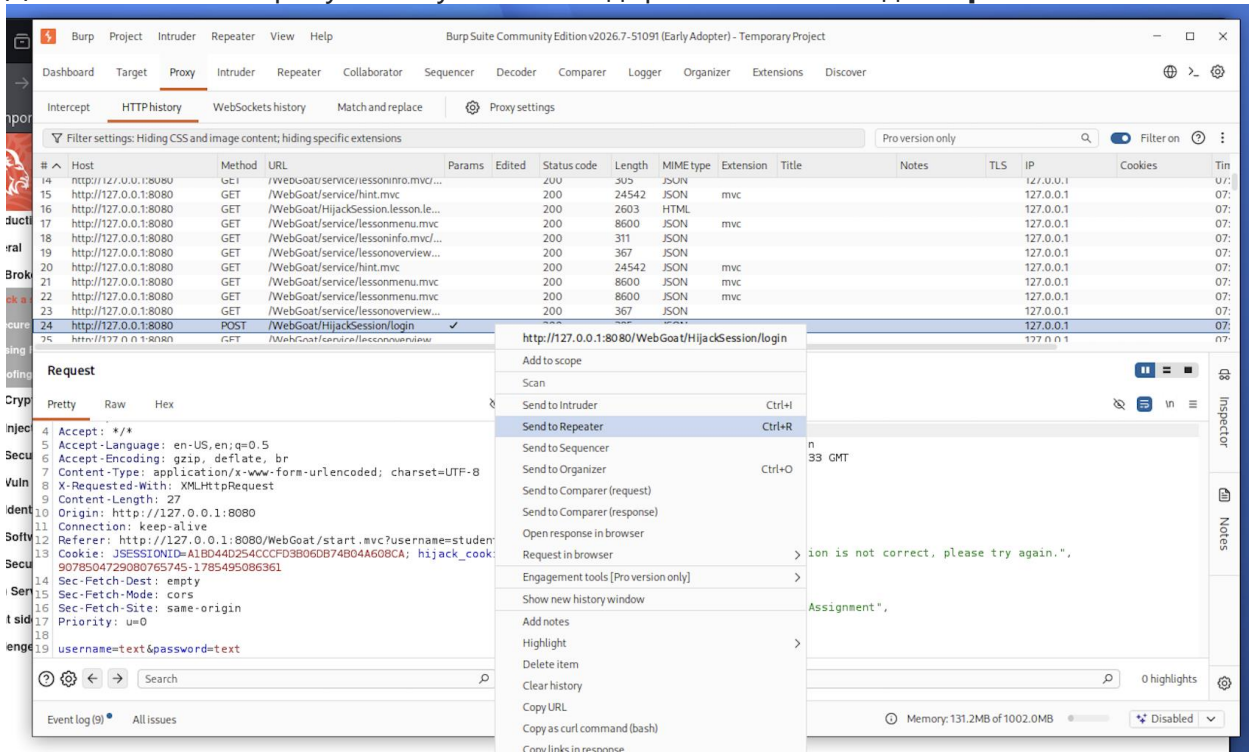
натискаємо "Access", отримуємо помилку та повертаємось до **burpsuit**, де маємо передивитись вкладинку **HTTPHistory**:



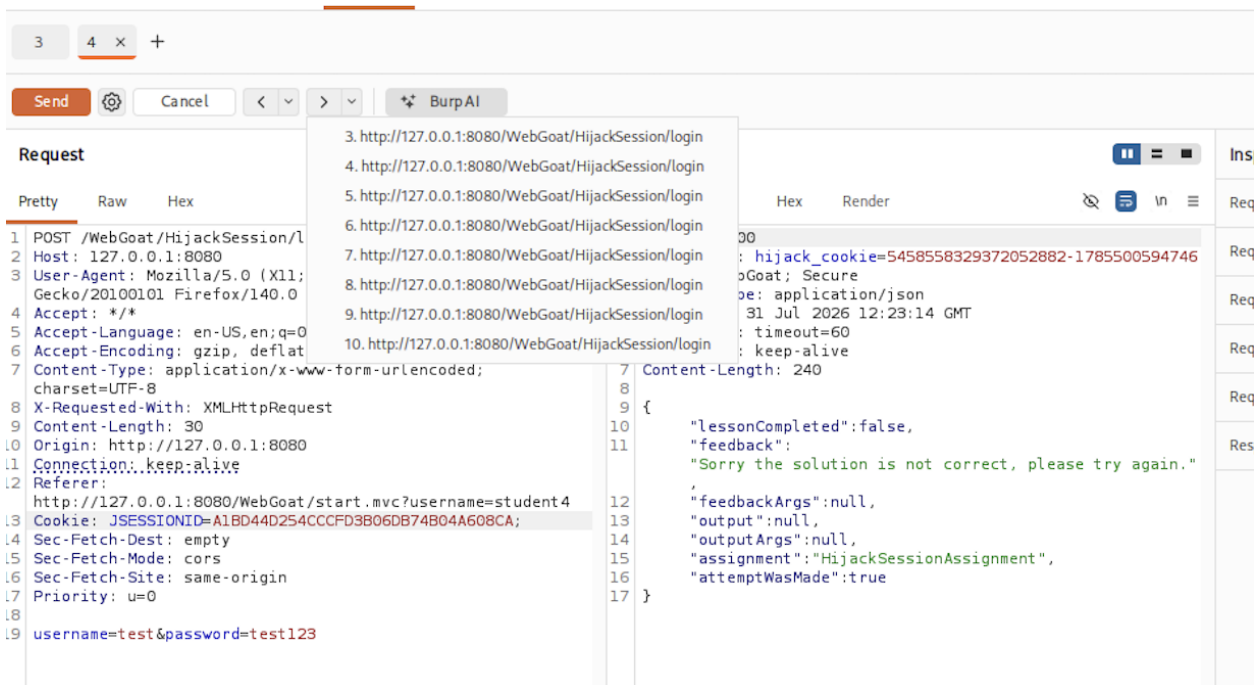
Далі шукаємо відповідь **POST**, в якій міститься інформація про цю помилку (ключ - це зміст **hijack_cookie**, яку нам і потрібно "передбачити"):



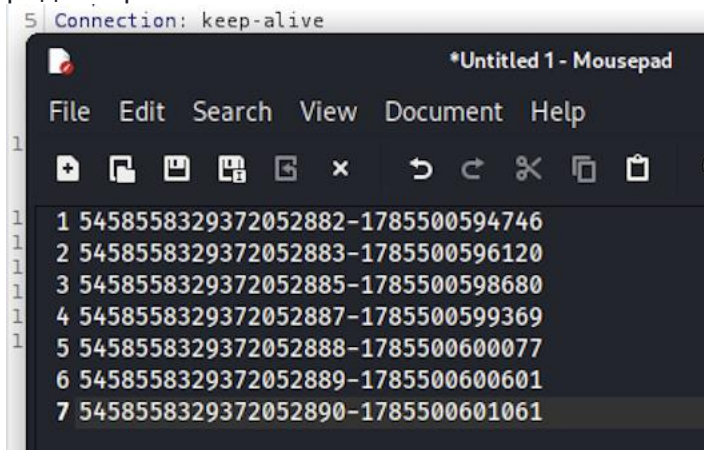
Далі натискаємо праву кнопку миші та відправляємо запит до **Repeater**:



В **Repeater** робимо декілька повторів (в цьому прикладі може вистачити 5+ запитів):

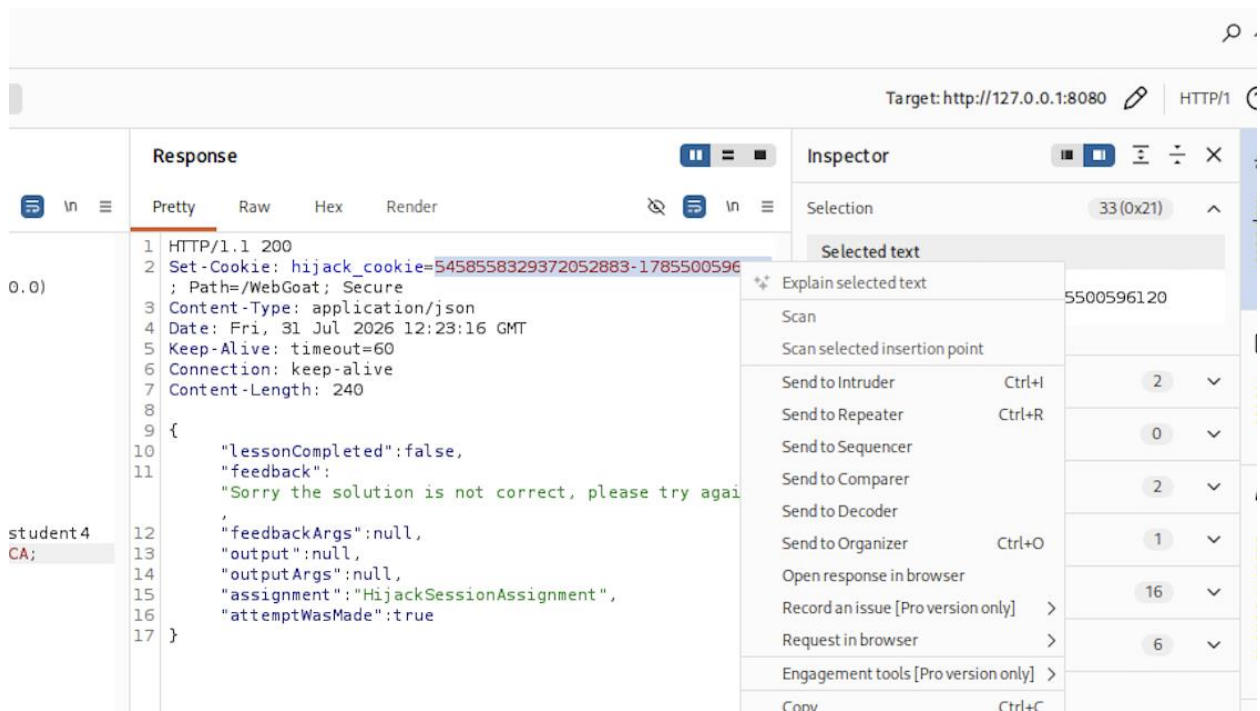


та бачимо, як змінюється значення **hijack_cookie**, копіюємо ці значення в текстовий редактор:



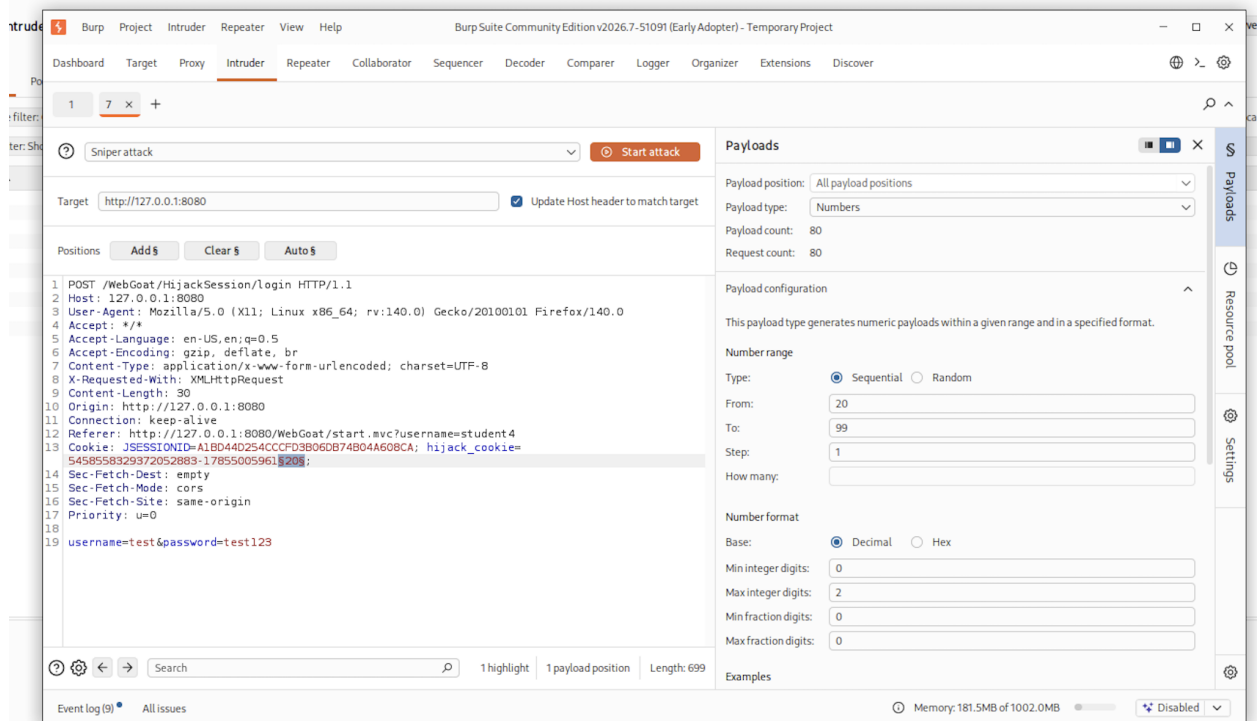
Навіть поверхневий аналіз дає висновок про логіку створення куки, перша частина відповідає за сесію користувача, а друга скоріш за все є **часовою міткою**, або **timestamp**.

Бачимо, що між 2 5458558329372052883 та 5458558329372052885 може бути сесія 5458558329372052884. Передаємо запит з сесією 5458558329372052883 до **intruder**:



Далі в **Intruder** формуємо свій запит відповідним чином:

- додаємо **hijack_cookie** 5458558329372052883-1785500596122, останні дві цифри будемо змінювати, такий собі лайтовий брутфорс



Запускаємо **start attack** та чекаємо результатів перебору, однак майже одразу отримуємо результат:

The screenshot displays the Burp Suite interface during an intruder attack. The top section shows the attack configuration: "17. Intruder attack of http://127.0.0.1:8080". Below this, the "Results" tab is active, showing a table of attack results. The table has columns for Request, Payload, Status code, Response received, Error, Timeout, Length, and Comment. The first row (index 0) shows a successful attack with a status code of 200 and a response received of 4. The bottom section of the interface provides a detailed view of the selected request and response. The "Request" tab is active, showing a POST request to "/WebGoat/HijackSession/login" with a status of 200. The "Response" tab is also active, showing the response body in JSON format. The response body contains the following JSON:

```
{
  "lessonCompleted": false,
  "feedback": "Sorry the solution is not correct, please try again.",
  "feedbackArgs": null,
  "output": null,
  "outputArgs": null,
  "assignment": "HijackSessionAssignment",
  "attemptWasMade": true
}
```

Чомусь у мене не вийшло, можедесь помилилась